

IA Aplicada com LLMs

Aula 08 — O agente e o estado

A terceira quase nunca é dita — e é a que decide

Um agente funciona porque **erra e se corrige**. Sem sinal de erro vindo do mundo, o laço não corrige nada: ele acumula passos sobre uma premissa errada, com a confiança intacta.

Tarefa	Feedback?
Corrigir bug — o teste passa ou falha	sim , binário
Investigar despesa — os valores batem ou não	sim , parcial
Escrever texto persuasivo	não → não é agente

Parte 1

O problema

O laço da Aula 03 guarda tudo aqui

```
mensagens = [  
    {"role": "system", "content": SYSTEM},  
    {"role": "user", "content": pergunta},  
]
```

Estamos na mensagem 40. Respondam, olhando **só para a lista**:
quantos passos? quantos tokens? quanto custou? qual era o objetivo?
que ferramenta falhou? alguma chamada se repetiu? **por que parou?**

As respostas

Pergunta	Onde está?
Quantos passos já foram dados?	dá para <i>inferir</i> , contando
Quantos tokens foram gastos?	não está — o <code>usage</code> foi descartado
Quanto custou?	idem
Qual era o objetivo?	soterrado na mensagem 2
A ferramenta X já falhou?	como texto , dentro de <code>content</code>
Alguma chamada se repetiu?	varrendo e reparseando JSON
Por que o agente parou?	em lugar nenhum

O diagnóstico: mensagens [] é o estado E é o transporte

Ninguém guarda o estado de um pedido dentro do JSON que vai para o cliente HTTP.

Mas é exatamente isso que o laço da Aula 03 faz.

Parte 2

A virada

O objeto de estado

```
@dataclass
class Estado:
    objetivo: str # imutável, a âncora
    passos: list[Passo] = field(default_factory=list)
    tokens_gastos: int = 0
    custo_estimado: float = 0.0
    ferramentas_ativas: list[str] = field(default_factory=list)
    termino: Termino | None = None
    resposta: str | None = None
    historico: list[dict] = field(default_factory=list) # DERIVADO
```

É a definição de agente da Aula 01 (§1.2) virando `dataclass`.

objetivo é campo, não a primeira mensagem → sobrevive à compaction.

passos guarda ferramenta e argumentos em campos, não em texto.

termino é obrigatório → quem para sem dizer por quê é indepurável.

A inversão

```
def montar_mensagens(estado: Estado) -> list[dict]:  
    """A lista de mensagens é uma VISÃO do estado."""  
    msgs = [{"role": "system", "content": SYSTEM},  
            {"role": "user", "content": estado.objetivo}]  
    msgs.extend(estado.historico)  
    if estado.n_passos % REANCORAR_A_CADA == 0:  
        msgs.append({"role": "user",  
                     "content": f"Lembrete do objetivo: {estado.objetivo}"})  
    return msgs
```

O histórico deixou de ser a verdade. Virou **formato de transporte**.

O laço, agora

```
def rodar(objetivo: str, orcamento: Orcamento) -> Estado:
    estado = Estado(objetivo=objetivo, ferramentas_ativas=list(FERRAMENTAS))
    while True:
        if (motivo := orcamento.excedido(estado)):
            estado.termino = Termino.ORCAMENTO
            return estado
    ...
```

1. Devolve **Estado**, não **str**
2. **while True** com **saídas nomeadas**, no lugar de **for range()**
3. **historico** é campo → pode ser **reescrito**

Isso não deixa o agente mais inteligente. Deixa o agente **observável**.

O que o estado destrava

Salvaguarda	Precisa de
Orçamento em 4 moedas	tokens, custo, passos, tempo
Motivo de término	<code>termino</code>
Erro recuperável × fatal	<code>Passo.erro</code>
Detecção de laço	ferramenta + argumentos separados
Reancoragem	<code>objetivo</code> fora do histórico
Checkpoint	estado serializável
Compaction	<code>historico</code> isolado

Sete salvaguardas, **um** pré-requisito.

Parte 3

Duas consequências

Ferramentas por fase

```
FASES = {  
    "coleta": ["buscar_despesa", "consultar_politica"],  
    "analise": ["consultar_politica", "consultar_historico"],  
    "registro": ["registrar_parecer"], # escrita, e SÓ ela  
}
```

Na fase `coleta`, `registrar_parecer` **não existe** para o modelo.

Instrução ele pode ignorar; ferramenta não declarada, não.

E o custo vem junto: cada declaração viaja em **todas** as voltas —
20 ferramentas × ~120 tokens = ~2.400 tokens por volta.

Restrição por arquitetura vence restrição por prompt.

Humano no laço — as duas versões

```
if chamada.function.name == "registrar_parecer":  
    if input("confirma? [s/N] ") != "s":      # implementação inadequada  
        ...
```

Prende um processo esperando gente, não sobrevive a reinício,
não existe quando o agente roda atrás de uma fila.

```
if nome in EXIGE_CONFIRMACAO:                    # a versão certa  
    estado.termino = Termino.HUMANO  
    estado.pendencia = {"ferramenta": nome, "argumentos": argumentos}  
    salvar_checkpoint(estado)  
    raise PausaParaHumano(estado)
```

Confirmação é **uma das quatro formas de terminar**: o agente para,
grava e devolve o controle. Retoma horas depois, sem repetir passo.

E tem preço

Cada ponto de confirmação **mata um pedaço da autonomia** que justificava o agente.

Um agente que pede confirmação a cada passo é um assistente de digitação caro.

Confirmação vai onde o erro é irreversível. Em nenhum outro lugar.

Parte 4

O checkpoint

Quatro linhas, e o que elas destravam

```
def salvar_checkpoint(estado: Estado) -> None:
    CHECKPOINTS.mkdir(exist_ok=True)
    (CHECKPOINTS / f"{estado.execucao_id}.json").write_text(
        json.dumps(asdict(estado), ensure_ascii=False, indent=2))
```

Confirmação assíncrona — o processo encerra; a aprovação chega depois.

Retomada após queda — não repete os 9 passos **nem seus efeitos**.

Depuração por reprodução — carregar um estado ruim e continuar dali.

O `asdict` só existe porque o estado é uma **dataclass** — com `mensagens []`, não há o que serializar.

O arquivo, por extenso

```
{
  "execucao_id": "exec-0a1f",
  "passos": [ {"indice": 0, "ferramenta": "consultar_historico",
               "argumentos": {"funcionario": "F-088"},
               "resultado": {"viagens_2026": 2} }, ... ],
  "tokens_gastos": 1150,
  "termino": "HUMAN0",
  "motivo": "registrar_parecer exige aprovação",
  "pendencia": {"ferramenta": "registrar_parecer",
                 "argumentos": {"despesa": "D-4102", "veredito": "aprovado"}}
}
```

O processo terminou, e ninguém está esperando na memória. O que sustenta a retomada é este arquivo, e não um laço bloqueado.

Retomar não é recomeçar

carregado: 2 passos já executados, 1150 tokens já gastos

aprovado -> executa o passo 2, e SÓ ele
registrar_parecer(D-4102) -> P-0993

término: CONCLUIDO passos: $2 + 1 = 3$

Os **1.150 tokens** não são gastos de novo — e, o que importa mais, nenhuma consulta é repetida e **nenhuma escrita é duplicada**.

Sem o checkpoint, "aprovar" seria reexecutar do zero. E a aprovação do humano recairia sobre uma execução **diferente** da que ele analisou.

A armadilha: persistir **DEPOIS** de executar, nunca antes

Se o checkpoint grava a **intenção** e o processo cai antes da execução, a retomada **refaz a ação** — e se a ação é escrita, ela acontece duas vezes.

A fresta que sobra — cair **entre** executar e gravar — é fechada pela **chave de idempotência**: o id da escrita vem do **conteúdo**, não da tentativa, então a segunda encontra a primeira.

As duas salvaguardas trabalham juntas. Nenhuma basta sozinha.

E o que ele não resolve

O arquivo descreve **uma** execução, e serve para retomá-la.

Amanhã chega outra tarefa, e a pergunta muda: "*o que já se aprendeu antes que ajude nesta?*". **O checkpoint não responde.** Não por estar incompleto — **ele está íntegro**

—,

mas porque não há **por onde perguntar** a ele.

Serializar todas as execuções multiplica o arquivo **sem criar a consulta.**

É a pergunta que abre a próxima nota.